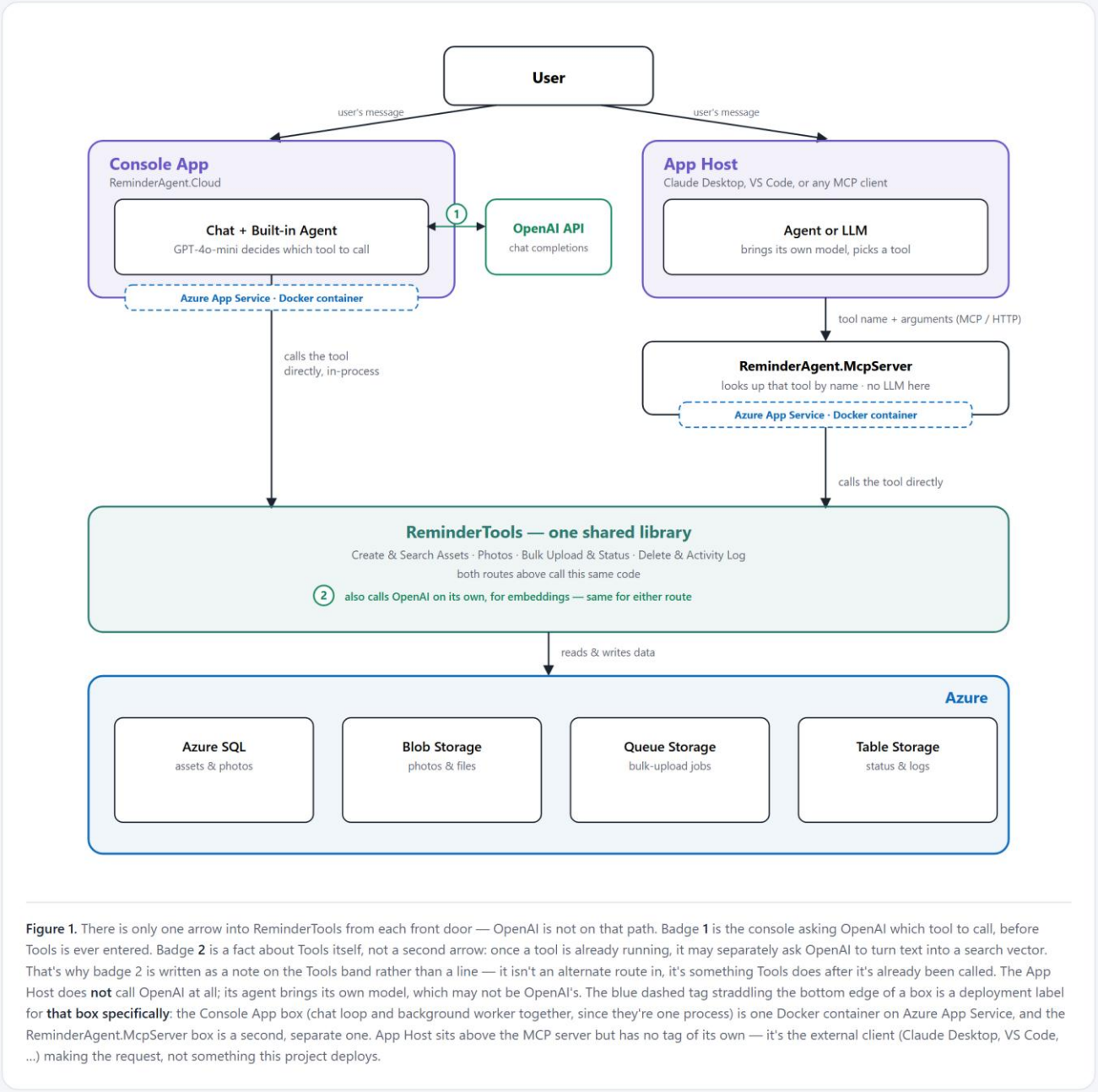


Figure 1. There is only one arrow into ReminderTools from each front door — OpenAI is not on that path. Badge **1** is the console asking OpenAI which tool to call, before Tools is ever entered. Badge **2** is a fact about Tools itself, not a second arrow: once a tool is already running, it may separately ask OpenAI to turn text into a search vector. That's why badge 2 is written as a note on the Tools band rather than a line — it isn't an alternate route in, it's something Tools does after it's already been called. The App Host does **not** call OpenAI at all; its agent brings its own model, which may not be OpenAI's. The blue dashed tag straddling the bottom edge of a box is a deployment label for **that box specifically**: the Console App box (chat loop and background worker together, since they're one process) is one Docker container on Azure App Service, and the ReminderAgent.McpServer box is a second, separate one. App Host sits above the MCP server but has no tag of its own — it's the external client (Claude Desktop, VS Code, ...) making the request, not something this project deploys.

The console has its own built-in agent and calls tools directly. An MCP client instead sends its request over MCP/HTTP to the MCP server; the server itself has no agent — it just runs the tool the client already chose. Either way, there is exactly **one** path into the tool library — OpenAI is a separate, independent conversation on the side.

02 — COMPONENT

Console app: one process, two things happening

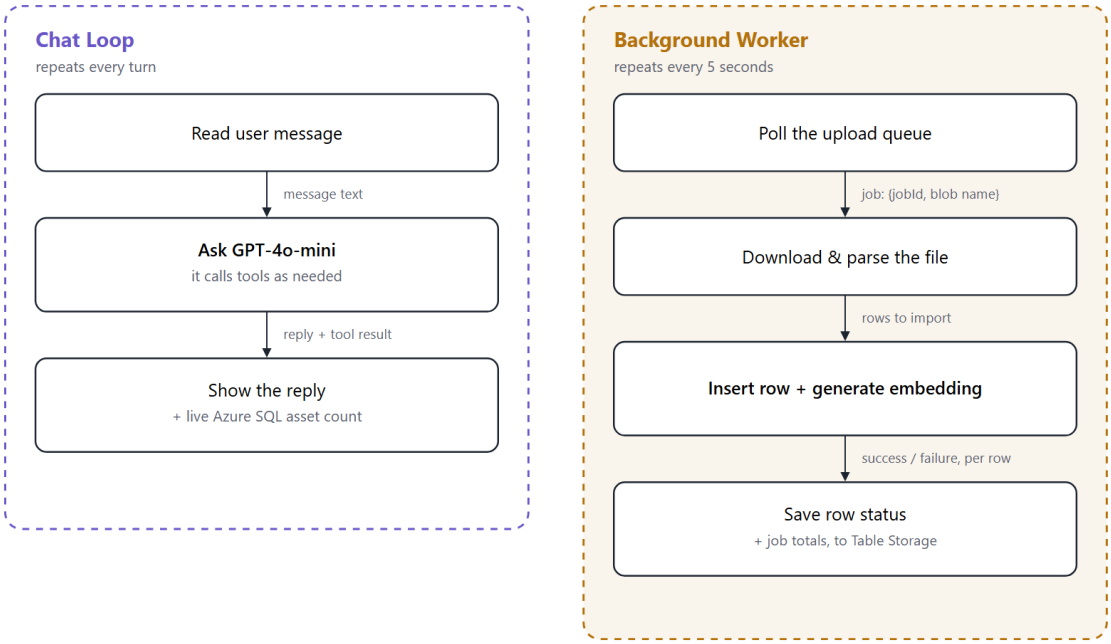


Figure 2. Both loops live in the same console process but run independently: the chat loop blocks on user input each turn, the background worker just sleeps five seconds between polls. Because neither waits on the other, a slow bulk import never freezes the chat, and a long conversation never delays the queue — and each imported row's outcome is saved as soon as that row finishes, not held until the whole file is done.

The console runs a chat loop for the person typing, and a background worker for bulk uploads, at the same time, in the same process.

03 — COMPONENT

MCP server: looks up the tool, doesn't choose it

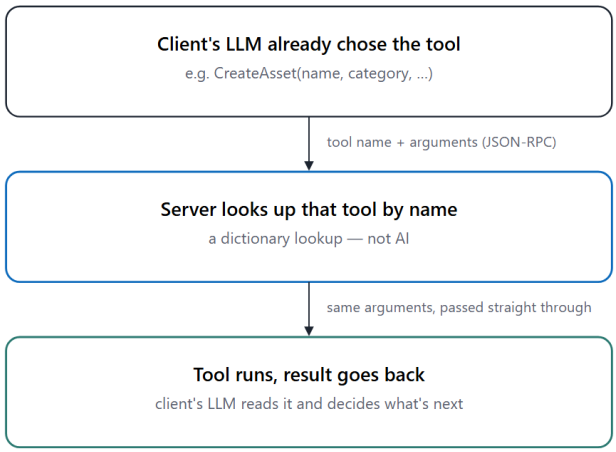


Figure 3. "Looks up" in the middle box means matching an exact name, the way a dictionary lookup does — not understanding. The client already did the understanding before it ever sent the request.

The client's own LLM is what reads the user's request and picks a tool — the server never sees plain language. It only ever sees an exact tool name and gets the arguments ready.

04 — WORKFLOW

Bulk upload: reply now, finish later

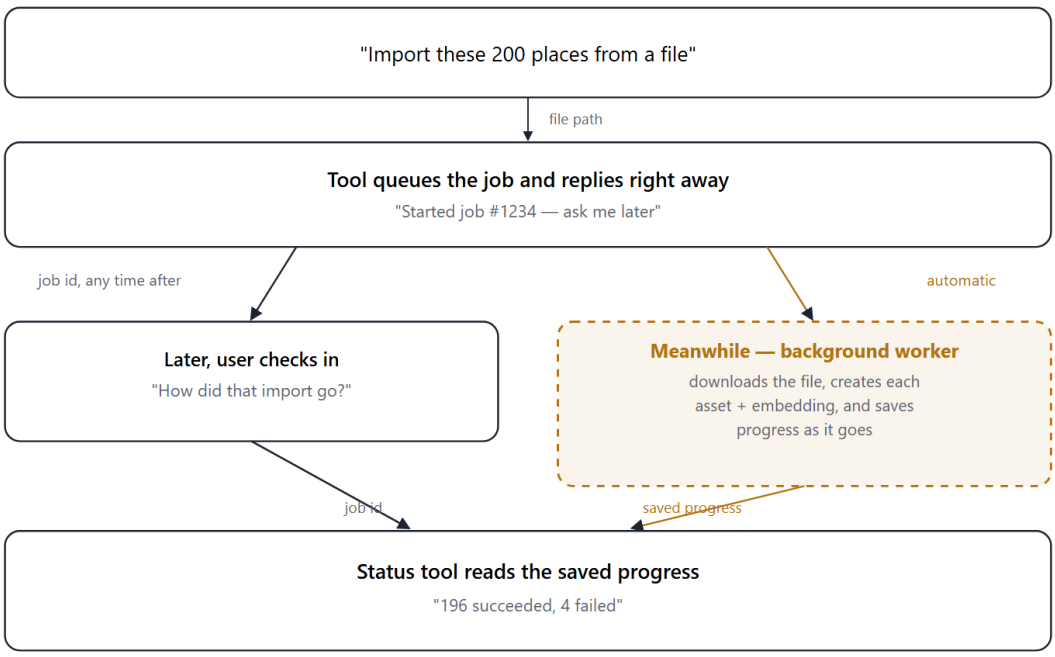


Figure 4. The job id is the thread that ties the quick reply to the slow work happening behind it.

Importing hundreds of rows takes too long to make the user wait. So the tool replies immediately, and the real work happens in the background.